

5.模板

函数模板

C++

```
1  #include<iostream>
2  using namespace std;
3  template<typename T>
4  T abs(T x){
5      return x<0?-x:x;
6  }
7  int main()
8  {
9      int n=-5;
10     double d=-5.5;
11     cout<<abs(n)<<endl;
12     cout<<abs(d)<<endl;
13     return 0;
14 }
```

排序函数模板

C++

```
1  #include<iostream>
2  #include<iomanip> // 用于设置输出格式
3  using namespace std;
4
5  // 模板函数：冒泡排序
6  template <typename T>
7  void bubbleSort(T a[], int n) {
8      for (int i = 0; i < n - 1; ++i) {
9          for (int j = 0; j < n - i - 1; ++j) {
10             if (a[j] > a[j + 1]) {
11                 T temp = a[j];
12                 a[j] = a[j + 1];
13                 a[j + 1] = temp;
14             }
15         }
16     }
17 }
18 int main() {
19     const int arraySize = 10; // 数组大小
20     int a[arraySize] = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1}, i;
21
22     // 显示原始整型数组
23     cout << "Integer data items in original order\n";
24     for (i = 0; i < arraySize; ++i)
25         cout << setw(6) << a[i]; // 设置字段宽度为6
26
27     bubbleSort(a, arraySize); // 对整型数组进行排序
28
29     // 显示排序后的整型数组
30     cout << "\nInteger data items in ascending order\n";
31     for (i = 0; i < arraySize; ++i)
32         cout << setw(6) << a[i];
33
34     cout << "\n\n";
35
36     // 初始化双精度浮点型数组
37     double b[arraySize] = {10.1, 9.9, 8.8, 7.7, 6.6, 5.5, 4.4, 3.3, 2.2, 1.1};
38
39     // 显示原始双精度浮点型数组
40 }
```

```

41     cout << "double point data items in original order\n";
42     for (i = 0; i < arraySize; ++i)
43         cout << setw(6) << b[i];
44
45     bubbleSort(b, arraySize); // 对双精度浮点型数组进行排序
46
47     // 显示排序后的双精度浮点型数组
48     cout << "\ndouble point data items in ascending order\n";
49     for (i = 0; i < arraySize; ++i)
50         cout << setw(6) << b[i];
51
52     cout << endl;
53
54     return 0;
55 } // end main

```

重载函数模板

```

1  #include<iostream>
2  #include<iomanip> // 用于设置输出格式
3  using namespace std;
4
5  // 模板函数：冒泡排序
6  template <typename T>
7  void bubbleSort(T a[], int n) {
8      // 外层循环控制遍历次数，每次确定一个元素的最终位置
9      for (int i = 0; i < n - 1; ++i) {
10         // 内层循环控制相邻元素比较和交换
11         for (int j = 0; j < n - i - 1; ++j) {
12             // 如果前一个元素大于后一个元素，则交换它们
13             if (a[j] > a[j + 1]) {
14                 T temp = a[j];
15                 a[j] = a[j + 1];
16                 a[j + 1] = temp;
17             }
18         }
19     }
20 }
21
22 int main() {
23     const int arraySize = 10; // 数组大小
24     int a[arraySize] = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1}, i;
25
26     // 显示原始整型数组
27     cout << "Integer data items in original order\n";
28     for (i = 0; i < arraySize; ++i)
29         cout << setw(6) << a[i]; // 设置字段宽度为6
30
31     bubbleSort(a, arraySize); // 对整型数组进行排序
32
33     // 显示排序后的整型数组
34     cout << "\nInteger data items in ascending order\n";
35     for (i = 0; i < arraySize; ++i)
36         cout << setw(6) << a[i];
37
38     cout << "\n\n";
39
40     // 初始化双精度浮点型数组
41     double b[arraySize] = {10.1, 9.9, 8.8, 7.7, 6.6, 5.5, 4.4, 3.3, 2.2, 1.1};
42
43     // 显示原始双精度浮点型数组
44     cout << "double point data items in original order\n";
45     for (i = 0; i < arraySize; ++i)
46         cout << setw(6) << b[i];
47
48     bubbleSort(b, arraySize); // 对双精度浮点型数组进行排序
49
50     // 显示排序后的双精度浮点型数组
51     cout << "\ndouble point data items in ascending order\n";

```

C++

```
52     for (i = 0; i < arraySize; ++i)
53         cout << setw(6) << b[i];
54
55     cout << endl;
56
57     return 0;
58 } // end main
59
```

C++ ▾

复制

...

非模板方式重载数组模板

C++

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  // 函数模板：打印整型数组
6  void printArray(const int* array, int count) {
7      // 遍历数组元素并输出
8      for (int i = 0; i < count && array[i] ≠ '\0'; i++) {
9          cout << array[i] << " ";
10     }
11     cout << endl;
12 }
13
14 // 函数模板：打印双精度浮点型数组
15 void printArray(const double* array, int count) {
16     // 遍历数组元素并输出
17     for (int i = 0; i < count && array[i] ≠ '\0'; i++) {
18         cout << array[i] << " ";
19     }
20     cout << endl;
21 }
22
23 // 函数模板：打印字符型数组
24 void printArray(const char* array, int count) {
25     // 遍历数组元素并输出
26     for (int i = 0; i < count && array[i] ≠ '\0'; i++) {
27         cout << array[i] << " ";
28     }
29     cout << endl;
30 }
31
32 // 函数模板：打印字符串数组
33 void printArray(const char** array, int count) {
34     // 遍历字符串数组元素并输出，每行输出4个字符串，右对齐
35     for (int i = 0; i < count && array[i] ≠ '\0'; i++) {
36         cout << setw(10) << right << array[i];
37         if ((i + 1) % 4 == 0 || i == count - 1) {
38             cout << endl;
39         }
40     }
41     cout << endl;
42 }
43
44 int main() {
45     // 定义数组大小
46     const int aCount = 5;
47     const int bCount = 7;
48     const int cCount = 6;
49     const int sCount = 8;
50
51     // 初始化数组
52     int a[aCount] = {1, 2, 3, 4, 5};
53     double b[bCount] = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
54     char c[cCount] = "HELLO"; // 字符数组，第6个位置存储空字符
55     const char* strings[sCount] = {"one", "two", "three", "four",
56                                     "five", "six", "seven", "eight"};
57
58     // 打印整型数组
```

```

59     cout << "Array a contains:\n";
60     printArray(a, aCount);
61
62     // 打印双精度浮点型数组
63     cout << "\nArray b contains:\n";
64     printArray(b, bCount);
65
66     // 打印字符型数组
67     cout << "\nArray c contains:\n";
68     printArray(c, cCount);
69
70     // 打印字符串数组
71     cout << "\nArray strings contains:\n";
72     printArray(strings, sCount);
73
74     return 0;
75 } // end main

```

ISEQUALTO 模板

```

1  #include<iostream>
2  #include<cmath>
3  #include<iomanip>
4  using namespace std;
5  template<typename T>
6  bool isEqualTo(T &a, T &b) {
7      return a==b;}
8  class SomeClass {
9  private:
10     double n1, n2;
11 public:
12     SomeClass(double nn1, double nn2) : n1(nn1), n2(nn2) {}
13     SomeClass() {
14         n1 = 0;
15         n2 = 0;
16     }
17     bool operator==(const SomeClass &other)
18     const{
19         return ((n1==other.n1)&&(n2==other.n2));
20     }
21     friend ostream& operator<<(ostream& output, const SomeClass& obj) {
22         output << "(" << obj.n1 << "," << obj.n2 << ")";
23         return output;
24     }
25
26     friend istream& operator>>(istream& input, SomeClass& obj) {
27         double nn1, nn2;
28         input >> nn1 >> nn2;
29         obj.n1 = nn1;
30         obj.n2 = nn2;
31         return input;
32     }
33 };
34 //StudybarCommentBegin
35 int main()
36 {
37     int a;    // integers used for
38     int b;    // testing equality
39
40     // test if two ints input by user are equal
41     //  cout << "Enter two integer values: ";
42     cin >> a >> b;
43
44     char c;   // chars used for
45     char d;   // testing equality
46
47     // test if two chars input by user are equal
48     //  cout << "\nEnter two character values: ";
49     cin>> c >> d;

```

C++

```

50
51     double e;    // double values used for
52     double f;    // testing equality
53
54     // test if two doubles input by user are equal
55 //     cout << "\nEnter two double values: ";
56     cin >> e >> f;
57     SomeClass g( 1, 1.1 ); // SomeClass objects used
58     SomeClass h; // for testing equality
59     cin >> h;
60     cout << a << " and " << b << " are "
61     << ( isEqualTo( a, b ) ? "equal" : "not equal" ) << '\n';
62     cout << c << " and " << d << " are "
63     << ( isEqualTo( c, d ) ? "equal" : "not equal" ) << '\n';
64     cout << e << " and " << f << " are "
65     << ( isEqualTo( e, f ) ? "equal" : "not equal") << '\n';
66     // test if two SomeClass objects are equal
67     // uses overloaded << operator
68     cout << "The class objects " << g << " and " << h << " are "
69     << ( isEqualTo( g, h ) ? "equal" : "not equal" ) << '\n';
70     return 0;
71 } // end main
72
73 //StudybarCommentEnd

```

类模板

```

1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;
4  struct Student {
5      int id;        //学号
6      float gpa;     //平均分
7  };
8  template <class T> //模板声明语法
9  class Store { //类模板：实现对任意类型数据进行存取
10 private:
11     T item; // item用于存放任意类型的数据
12     //任意类型
13     bool haveValue; // haveValue标记item是否已被存入内容
14 public:
15     Store();
16     T &getElem(); //提取数据函数
17     void putElem(const T &x); //存入数据函数
18 };
19
20 template <class T>
21 Store<T>::Store(): haveValue(false) { }
22 template <class T>
23 T &Store<T>::getElem() {
24     //如试图提取未初始化的数据，则终止程序
25     if (!haveValue) {
26         cout << "No item present!" << endl;
27         exit(1); //使程序完全退出，返回到操作系统。
28     }
29     return item; // 返回item中存放的数据
30 }
31 template <class T>
32 void Store<T>::putElem(const T &x) {
33     // 将haveValue 置为true，表示item中已存入数值
34     haveValue = true;
35     item = x; // 将x值存入item
36 }
37
38 int main() {
39     Store<int> s1, s2;
40     s1.putElem(3);
41     s2.putElem(-7);
42     cout << s1.getElem() << " " << s2.getElem() << endl;

```

C++

```

43
44     Student g = { 1000, 23 };
45     Store<Student> s3;
46     s3.putElem(g);
47     cout << "The student id is " << s3.getElem().id << endl;
48
49     Store<double> d;
50     cout << "Retrieving object D... ";
51     cout << d.getElem() << endl;
52     //d未初始化，执行函数D.getElement()时导致程序终止
53     return 0;
54 }

```

整数合集

C++

```

1  #include <iostream> // 引入标准输入输出流库
2  #include <algorithm> // 引入算法库，用于排序
3  #include <vector> // 引入向量库，用于存储集合元素
4  using namespace std; // 使用标准命名空间
5  class Cassemblage { // 定义 Cassemblage 类，表示整数集合
6  private:
7      vector<int> elements; // 私有变量，存储集合元素的向量
8  public:
9      void Set(int* arr, int count) { // 成员函数 Set，用于设置集合元素
10         elements.resize(count); // 调整元素向量的大小为 count
11         for (int i = 0; i < count; ++i) { // 遍历元素数组 arr
12             elements[i] = arr[i]; // 将数组元素赋值给元素向量中对应位置
13         }
14         sort(elements.begin(), elements.end()); // 对元素向量进行排序
15     }
16
17     void Show() const { // 成员函数 Show，用于显示集合元素
18         if (elements.empty()) { // 如果元素向量为空
19             cout << "empty" << endl; // 输出 "empty"
20         } else { // 如果元素向量不为空
21             for (int i = 0; i < elements.size(); ++i) { // 遍历元素向量
22                 cout << elements[i] << " "; // 输出每个元素并加上空格
23             }
24             cout << endl; // 输出换行
25         }
26     }
27
28     Cassemblage operator+(const Cassemblage& other) const { // 重载运算符+，表示并集
29         Cassemblage result = *this; // 创建一个临时集合，初始化为当前集合
30         for (int i = 0; i < other.elements.size(); ++i) { // 遍历另一个集合的元素向量
31             if (find(result.elements.begin(), result.elements.end(), other.elements[i]) == result.elements.end())
32         ) { // 如果当前元素不在临时集合中
33                 result.elements.push_back(other.elements[i]); // 将当前元素添加到临时集合中
34             }
35         }
36         sort(result.elements.begin(), result.elements.end()); // 对临时集合元素进行排序
37         return result; // 返回并集结果
38     }
39
40     Cassemblage operator&(const Cassemblage& other) const { // 重载运算符&，表示交集
41         Cassemblage result; // 创建一个空集合
42         for (int i = 0; i < elements.size(); ++i) { // 遍历当前集合的元素向量
43             if (find(other.elements.begin(), other.elements.end(), elements[i]) != other.elements.end()) { // 如
44         果当前元素也在另一个集合中
45                 result.elements.push_back(elements[i]); // 将当前元素添加到交集集合中
46             }
47         }
48         return result; // 返回交集结果
49     }
50
51     Cassemblage operator-(const Cassemblage& other) const { // 重载运算符-，表示差集
52         Cassemblage result; // 创建一个空集合
53         for (int i = 0; i < elements.size(); ++i) { // 遍历当前集合的元素向量
54             if (find(other.elements.begin(), other.elements.end(), elements[i]) == other.elements.end()) { // 如

```



```
52 果当前元素不在另一个集合中
    result.elements.push_back(elements[i]); // 将当前元素添加到差集集合中
53     }
54 }
55 return result; // 返回差集结果
56 }
57 };
58
59 int main(int argc, char* argv[]) { // 主函数入口
60     Cassemlage z1, z2, x1, x2, x3; // 定义五个整数集合对象
61     int i, n1, n2, a1[1000], a2[1000]; // 定义整数和两个整数数组
62
63     cin >> n1; // 从标准输入流读取一个整数，表示集合 z1 的元素个数
64     for(i = 0; i < n1; i++) { // 循环读取 n1 个整数，存入数组 a1 中
65         cin >> a1[i];
66     }
67     z1.Set(a1, n1); // 调用集合对象 z1 的 Set 方法，设置集合元素
68
69     cin >> n2; // 从标准输入流读取一个整数，表示集合 z2 的元素个数
70     for(i = 0; i < n2; i++) { // 循环读取 n2 个整数，存入数组 a2 中
71         cin >> a2[i];
72     }
73     z2.Set(a2, n2); // 调用集合对象 z2 的 Set 方法，设置集合元素
74
75     x1 = z1 + z2; // 计算并集并赋值给 x1
76     x1.Show(); // 输出并集
77
78     x2 = z1 & z2; // 计算交集并赋值给 x2
79     x2.Show(); // 输出交集
80
81     x3 = z1 - z2; // 计算差集并赋值给 x3
82     x3.Show(); // 输出差集
83     return 0; // 返回退出状态
84 }
85
```